

1) Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

```
#include <stdio.h>

int main() {

    int n;

    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++) {

        scanf("%d", &arr[i]);

    }

    int low = 0;

    int high = n - 1;

    int firstZero = n;

    while (low <= high) {

        int mid = (low + high) / 2;

        if (arr[mid] == 0) {

            firstZero = mid;

            high = mid - 1;

        } else {

            low = mid + 1;

        }

    }

    printf("%d\n", n - firstZero);

    return 0;

}
```

2) Given an array **nums** of size **n**, return *the majority element*.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: nums = [3,2,3]

Output: 3

Example 2:

Input: nums = [2,2,1,1,1,2,2]

Output: 2

```
#include <stdio.h>

int main() {
    int n;

    scanf("%d", &n);

    int num[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &num[i]);
    }

    int c = num[0];
    int count = 1;

    for (int i = 1; i < n; i++) {
        if (num[i] == c) {
            count++;
        } else {
            count--;
        }

        if (count == 0) {
            c = num[i];
            count = 1;
        }
    }

    printf("%d\n", c);

    return 0;
}
```

3) Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

```
#include <stdio.h>

int findFloor(int arr[], int low, int high, int x) {
    if (low > high) {
        return -1;
    }
    if (x >= arr[high]) {
        return arr[high];
    }
    int mid = low + (high - low) / 2;
    if (arr[mid] == x) {
        return arr[mid];
    }
    if (mid > 0 && arr[mid - 1] <= x && x < arr[mid]) {
        return arr[mid - 1];
    }
    if (x < arr[mid]) {
        return findFloor(arr, low, mid - 1, x);
    }
    return findFloor(arr, mid + 1, high, x);
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int arr[n];
```

```

for (int i = 0; i < n; i++) {
    if (scanf("%d", &arr[i]) != 1) return 0;
}

int x;

if (scanf("%d", &x) != 1) return 0;

int floorValue = findFloor(arr, 0, n - 1, x);

printf("%d\n", floorValue);

return 0;
}

```

4) Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

```

#include <stdio.h>

#include <stdlib.h>

int main() {

    int n, target;

    if (scanf("%d", &n) != 1 || n <= 0) {

        printf("Invalid array size.\n");

        return 1;

    }

    int *arr = (int *)malloc(n * sizeof(int));

    if (!arr) {

        printf("Memory allocation failed.\n");

        return 1;

    }
}

```

```

}
for (int i = 0; i < n; i++) {
    if (scanf("%d", &arr[i]) != 1) {
        printf("Invalid input.\n");
        free(arr);
        return 1;
    }
}
if (scanf("%d", &target) != 1) {
    printf("Invalid target value.\n");
    free(arr);
    return 1;
}
int left = 0;
int right = n - 1;
int found = 0;
int num1 = 0, num2 = 0;
while (left < right) {
    int sum = arr[left] + arr[right];
    if (sum == target) {
        num1 = arr[left];
        num2 = arr[right];
        found = 1;
        break;
    } else if (sum > target) {
        right--;
    } else {
        left++;
    }
}
if (found) {

```

```

        printf("%d\n%d\n", num1, num2);
    } else {
        printf("No\n");
    }
    free(arr);
    return 0;
}

```

5) Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no of elements in the list-n

The next n lines contain the elements.

Output:

Sorted list of elements

```

#include <stdio.h>

int main() {
    int n, i, j, low, high, top;
    int arr[100], stack[100];
    int pivot, temp;
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    top = -1;
    stack[++top] = 0;
    stack[++top] = n - 1;
    while (top >= 0) {
        high = stack[top--];
        low = stack[top--];
        pivot = arr[high];
        i = low - 1;
    }
}

```

```

for (j = low; j < high; j++) {
    if (arr[j] < pivot) {
        i++;

        temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}
temp = arr[i + 1];
arr[i + 1] = arr[high];
arr[high] = temp;
int p = i + 1;
if (p - 1 > low) {
    stack[++top] = low;
    stack[++top] = p - 1;
}
if (p + 1 < high) {
    stack[++top] = p + 1;
    stack[++top] = high;
}
}
for (i = 0; i < n; i++) {
    printf("%d", arr[i]);
    if (i < n - 1)
        printf(" ");
}
return 0;
}

```